

L-Systems Worksheet

Section 1 - The Basics

- Create a basic turtle graphics implementation. It should have the following features
 - display window in which the image will be drawn
 - store the current location and angle
 - draw a line from the current location at the current angle of a given length. (You might want to be able to change the length in the future). When the line is drawn, move the current location to the end of the line and do not change the angle.
 - turn/rotate left or right by a given angle.
 - a stack to which the current position and angle can be stored to or restored from. When restoring the turtle should be set to the position & angle at the top of the stack and that position/angle is removed from the stack.
 - Take an enstruction string consisting of F, +, -, [, and] and draw the shape it describes.
- Create a basic L-system expansion system. It should
 - specify an alphabet, axiom and production rules. NB this doesn't have to be customisable at run time, it can be hard-coded. But plan ahead because you will want to change this later.
 - expand a string using symbols from the alphabet and the production rules to create a new instruction string
- add buttons (or some form of control) to your turtle graphics application to
 - select & draw the initial axiom
 - carry out one full expansion of the currently drawn string and then draw it.
 - display the current iteration number (axiom = 0)

Section 2 - Tweaks

- Basic implementation improvements
 - as the number of iterations grows so does the image, implement a way to keep the full image visible
 - display the axiom, alphabet, production rules and current string
 - add a "Iterate X times" option where the user enters a number of expansions to be carried out
- Iteration safe guarding - the length of the string/number of replacements to be made can get very bit very quickly. Ensure your implementation doesn't lock up your PC. Consider warnings about length/time.
- Add a system to allow the user to add/create/edit the alphabet, axiom and production rules
- Add a system to allow the user to save/load the alphabet, axiom and production rules
- Stochastic L-System. Implement a stochastic l-system expansion system. You'll need to
 - develop a way to store production rules that include a probability
 - randomise which version of a rule a character is expanded using.
 - consider displaying/allowing the user to enter a seed for randomisation
- Consider colour options - all the same, selected colour, random? For example consider what it means if the next character is] or there isn't a next character.
- Consider adding characters which are never expanded but are intended to specify the need to draw something like a petal or a leaf.

Section 3 - Extensions

These are intentionally open and potentially very difficult. You are not expected to implement them all and I will be amazed if you do!

- Context-Sensitive L-System. Implement a context sensitive l-system expansion system. You'll need to
 - read up on context-based l-systems to ensure you understand them
 - develop a way to store production rules that include context
 - correctly evaluate which rule to follow
- Parametric L-System. Implement a parametric l-system expansion system. You'll need to
 - read up on parametric l-systems to ensure you understand them
 - develop a way to store production rules that include parameters
 - correctly evaluate which rule to follow
- Create a system, it can be a separate system, which will generate a forest/field of plants, each generated from an l-system. These can be trees, ferns, alien plants.
 - Recommended approach - identical plants first, then variations on a single l-system (parametric or with some randomisation of angles/lengths), mixed plants (one dominant and occasional different plants).
- Create an application which draws a cave or dungeon floor layout (i.e. a 2D map)

Circle Equation

For a circle with a centre (c_x, c_y) and radius r :

$$x = c_x + r \cos(\theta)$$

$$y = c_y + r \sin(\theta)$$

where:

$$0 \leq \theta \leq 2\pi$$

This might come in handy for finding the end point of a line that is r long, starting at (c_x, c_y) and along the angle θ .